IT418 Cloud Computing Project Report on

Predictive Serverless Cold-Start Optimizer

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

By

Rahul Saravanan (231IT054)

Shishir Hegde (231IT067)



Department of Information Technology

National Institute of Technology Karnataka, Surathkal

Mangalore – 575025

October, 2026

DECLARATION

We hereby declare that the IT418 Cloud Computing project report entitled *Predictive Serverless Cold-Start Optimizer*, submitted to the National Institute of Technology Karnataka, Surathkal, is a bona fide record of the work carried out by us. The material contained in this report has not been submitted to any other university or institution for the award of any degree.

(1) Rahul Saravanan (231IT054)

(2) Shishir Hegde (231IT067)

Department of Information Technology

Place: NITK, Surathkal – 575025

Date: October, 2026

Signatures of the students

CERTIFICATE

This is to certify that the IT418 Cloud Computing project report entitled *Predictive Serverless Cold-Start Optimizer*, submitted by Rahul Saravanan (231IT054) and Shishir Hegde (231IT067), is accepted as a B.Tech. project report in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology.

Bhawana Rudra
Designation: Assistant Professor
Department of Information Technology, NITK Surathkal

Chairman – DUGC
Signature with date and seal

ACKNOWLEDGEMENT

We express our sincere gratitude to the Department of Information Technology, National Institute of Technology Karnataka, Surathkal, for providing the academic environment and facilities required for this work. We thank our guide, Bhawana Rudra, for her guidance and feedback throughout the project. We also acknowledge the support of the faculty and staff who helped us refine the evaluation and presentation of the work.

Rahul Saravanan
Shishir Hegde

ABSTRACT

Serverless applications often spend a substantial part of their first-request latency loading dependencies. This project presents the Predictive Cold-Start Optimizer (PCSO), an application-level approach that combines static dependency analysis with runtime transition prediction. The static phase separates startup-critical imports from request-specific imports. The runtime phase learns likely endpoint transitions and preloads the predicted dependency closure before the next request arrives. Against an eager no-optimizer baseline, the evaluated system reduces blended code-loading latency from 31.36 ms to 4.77 ms and total response latency from 35.29 ms to 15.48 ms. These correspond to reductions of 84.8% and 56.1%, respectively. The static-only V1/FaaSLight stage provides a useful minimum-improvement reference but can shift heavy import cost into handler execution.

Keywords: serverless computing, cold start, dependency analysis, lazy loading, transition prediction, preloading.

CONTENTS

1 INTRODUCTION **1**

1.1 Overview 1

1.2 Motivation 1

1.3 Objectives 1

1.4 Scope and contributions 2

2 LITERATURE REVIEW **3**

2.1 Serverless cold starts 3

2.2 FaaSLight and static deferral 3

2.3 Research gap 3

2.4 Comparison of approaches 3

3 PROPOSED METHODOLOGY **5**

3.1 Static dependency analysis 5

3.2 Transition prediction 5

3.3 Preloading policy 6

4 RESULTS AND ANALYSIS **8**

4.1 Evaluation protocol 8

4.2 Latency comparison 9

4.3 Tier 3 analysis 10

4.4 Blended fleet analysis 11

5 CONCLUSIONS AND FUTURE WORK **12**

5.1 Conclusion 12

5.2 Limitations and future work 12

REFERENCES **12**

LIST OF FIGURES

3.1 PCSO combines a static dependency manifest with runtime prediction. The eager baseline loads the full dependency set, V1 loads only indispensable imports, and PCSO additionally warms the predicted next endpoint. 6

3.2 PCSO runtime loop: record a transition, predict the next endpoint, warm its dependency closure, and reuse the resident modules on the next invocation. 6

4.1 Mean code-loading and end-to-end response latency for the eager baseline, V1/FaaSLight, and final PCSO. 9

4.2 Reduction relative to the true eager baseline. PCSO extends the V1 improvement by removing shifted handler-side import cost. . . . 10

4.3 Resident memory and PCSO predictive cache-hit rate across the workload profiles. 11

LIST OF TABLES

4.1 Workload profiles used in the evaluation. 8
4.2 Compared runtime versions. 9
4.3 Mean latency comparison against the true no-optimizer baseline. . . 10

Chapter 1

INTRODUCTION

1.1 Overview

Function-as-a-Service platforms create execution environments on demand. A newly created environment must initialize application code before it can serve its first request. The initialization cost is particularly visible in services that include database clients, SDKs, model wrappers, or media-processing libraries. Loading every dependency eagerly is simple, but it makes optional functionality part of the critical path.

This project evaluates a progression from an eager no-optimizer baseline to the final Predictive Cold-Start Optimizer (PCSO). The comparison retains the static-only V1/FaaSLight stage as a minimum-improvement reference. The central objective is to reduce code-loading latency without moving the same cost into handler execution.

1.2 Motivation

Static dependency analysis can identify imports that are not needed for every request, but static information alone does not reveal when a deferred dependency will be needed. Runtime transition information provides that missing temporal signal. If endpoint A is commonly followed by endpoint B, the dependencies of B can be prepared while the system is processing A.

1.3 Objectives

(1) Classify dependencies into startup-critical and deferrable imports.

(2) Learn endpoint transitions using an online first-order model.

(3) Preload the predicted endpoint and its dependency closure.

(4) Compare the baseline, V1/FaaSLight, and PCSO across multiple workload profiles.

1.4 Scope and contributions

The work focuses on application-level initialization rather than changes to a cloud provider's scheduler or container runtime. The unit of optimization is an endpoint module and the set of local dependencies it can reach. This keeps the method portable: the same policy can be applied wherever the application controls its import boundary and has access to a warm execution environment.

The main contribution is the connection between two kinds of information. Static analysis provides a dependency manifest, while runtime observation provides ordering information between endpoint invocations. V1 uses only the first source of information. PCSO combines both sources and therefore has an opportunity to move heavy initialization work out of the next request's critical path.

Chapter 2

LITERATURE REVIEW

2.1 Serverless cold starts

A serverless cold start includes environment creation, application initialization, dependency loading, and handler execution. The relative contribution of these stages varies with application size and workload. For lightweight services, dependency loading can be a substantial fraction of total latency; for heavy services, a deferred model or processing library can dominate the handler path instead.

2.2 FaaSLight and static deferral

FaaSLight describes application-level cold-start optimization through dependency analysis and selective lazy loading [1]. Its key insight is that a dependency imported only on a conditional path need not be loaded before every request. This work adopts that principle as V1 and extends it with runtime transition prediction and preloading.

2.3 Research gap

Static deferral answers *what* can be delayed, but not *when* the delayed code will be needed. The remaining challenge is to avoid paying a large deferred import cost inside the next request. PCSO addresses this gap by combining static dependency information with a learned next-endpoint prediction.

2.4 Comparison of approaches

The three points in the evaluation have deliberately different responsibilities. The baseline establishes the cost of eager loading. V1 isolates the benefit of static analysis and lazy imports. PCSO measures the additional benefit of using sequence

information to warm the next endpoint. This separation is important because a large reduction in import time does not necessarily imply a large reduction in total response time when the import cost is merely moved into the handler.

Chapter 3

PROPOSED METHODOLOGY

3.1 Static dependency analysis

The analyzer inspects the source structure of each function. Imports at module scope are classified as indispensable because they are needed to construct the handler environment. Imports inside functions or conditional branches are classified as optional candidates for deferral. The classification changes the loading schedule, not the behavior of the handler.

The analysis is intentionally lexical rather than speculative. It does not attempt to infer the result of arbitrary program paths. A top-level import is therefore treated as startup-critical, while an import nested in a function or branch is eligible for deferred loading. Local application dependencies are retained in the manifest; standard-library and external dependencies are ignored by the runtime preloader when they are not part of the application workload.

3.2 Transition prediction

PCSO maintains a first-order Markov transition table. For an observed transition from endpoint A to endpoint B, it increments $N(A, B)$. The next-endpoint probability is estimated as

$$P(B \mid A) = \frac{N(A, B)}{\sum_x N(A, x)}. \tag{3.1}$$

The highest-probability endpoint is selected when its confidence reaches the configured threshold. If the confidence is low, the system falls back to the static-loading behavior.

Before any endpoint has been invoked, the transition table is empty. The first invocation therefore has no prediction and follows the V1 loading path. After the first completed transition, the corresponding row contains one observation. As more transitions are observed, the row estimates become more stable. Ties are

resolved deterministically by the predictor's ordering, and the confidence threshold prevents speculative warming when the observed evidence is weak.

For example, if the row for endpoint A contains seven observations of $A \to B$ and three observations of $A \to C$, then $P(B \mid A) = 0.70$ and $P(C \mid A) = 0.30$. PCSO warms B when the threshold is 0.35; it does not warm C as the top prediction for that row.

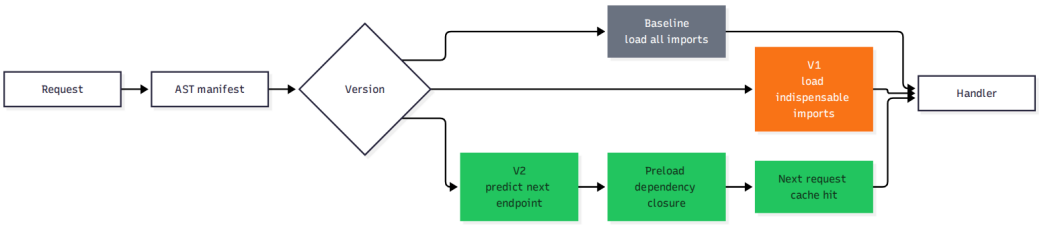


Figure 3.1: PCSO combines a static dependency manifest with runtime prediction. The eager baseline loads the full dependency set, V1 loads only indispensable imports, and PCSO additionally warms the predicted next endpoint.

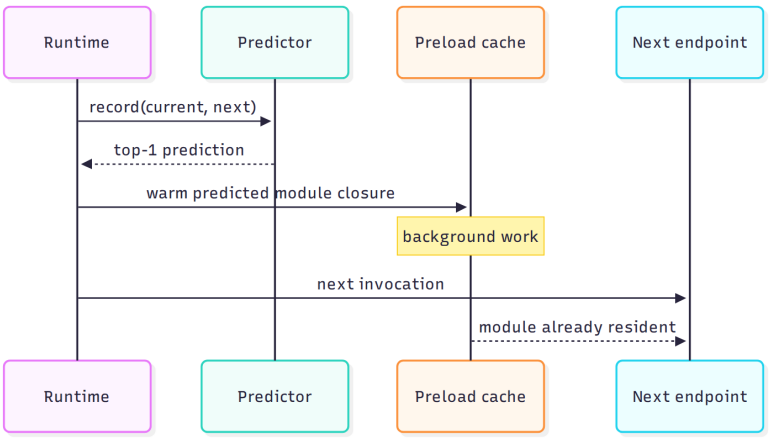


Figure 3.2: PCSO runtime loop: record a transition, predict the next endpoint, warm its dependency closure, and reuse the resident modules on the next invocation.

3.3 Preloading policy

After each invocation, PCSO records the completed transition and predicts the next endpoint. It then warms the predicted endpoint and its local dependency closure. The background preload time is tracked separately from request latency. If the next request matches the prediction, the module is already resident; otherwise, the request follows the static-loading path without changing functional

correctness.

The runtime sequence is therefore: (1) load the current endpoint, (2) execute its handler, (3) update the transition table, (4) select a predicted endpoint if the confidence threshold is met, and (5) import the predicted endpoint and its indispensable and optional local dependencies into the warm set. The measured response ends after step (2); step (5) is reported as background preparation. This distinction prevents the benchmark from presenting background work as if it were part of the current request's latency.

Chapter 4

RESULTS AND ANALYSIS

4.1 Evaluation protocol

The evaluation uses four workload profiles. Tier 1 represents lightweight CRUD and utility functions. Tier 2 represents medium enterprise services with authentication, validation, and persistence logic. Tier 3 represents heavy OCR, fraud-analysis, and media-processing pipelines. The blended profile samples 50% Tier 1, 35% Tier 2, and 15% Tier 3 activity.

Each tiered profile contains 24 invocations and the blended profile contains 32. Fixed random seeds make the sequences repeatable. Each version is evaluated in an isolated runtime state. The reported metrics are mean import latency, mean handler latency, mean total response latency, p95 latency, resident memory, and predictive cache-hit rate.

For an invocation i, total response latency is computed as

$$T_i = T_{\text{load},i} + T_{\text{handler},i}. \tag{4.1}$$

The percentage reduction of an optimized version M relative to the baseline is

$$R_M = 100 \times \frac{\overline{T}_{\text{baseline}} - \overline{T}_M}{\overline{T}_{\text{baseline}}}. \tag{4.2}$$

The p95 is the latency value at the 95th percentile of the ordered invocation samples. Cache-hit rate is the fraction of invocations whose endpoint module was already present in the predicted warm set.

Table 4.1: Workload profiles used in the evaluation.

Profile	**Size**	**Dominant behavior**
Tier 1	Light	CRUD, health, configuration, and utility operations.
Tier 2	Medium	Authentication, validation, database, and enterprise service logic.
Tier 3	Heavy	OCR, fraud evaluation, model, and media-processing dependencies.
Blended	Mixed	50% Tier 1, 35% Tier 2, and 15% Tier 3.

The baseline and V1 measurements isolate each invocation's application working set. PCSO follows the same cold-start isolation policy for unrelated modules but preserves the explicitly predicted working set. This is why the first PCSO call behaves like V1, while later calls can observe a cache hit.

Table 4.2: Compared runtime versions.

Version	Policy
Baseline	Eagerly loads indispensable and optional dependencies.
V1 / FaaSLight	Loads indispensable imports; optional imports remain handler-lazy.
PCSO	V1 policy plus transition prediction and dependency-closure preloading.

4.2 Latency comparison

The first pattern in Table 4.3 is that V1 consistently reduces code-loading time, but the total-latency effect depends on how much deferred work is exercised by the handler. PCSO reduces both terms when the transition model predicts the next endpoint successfully.

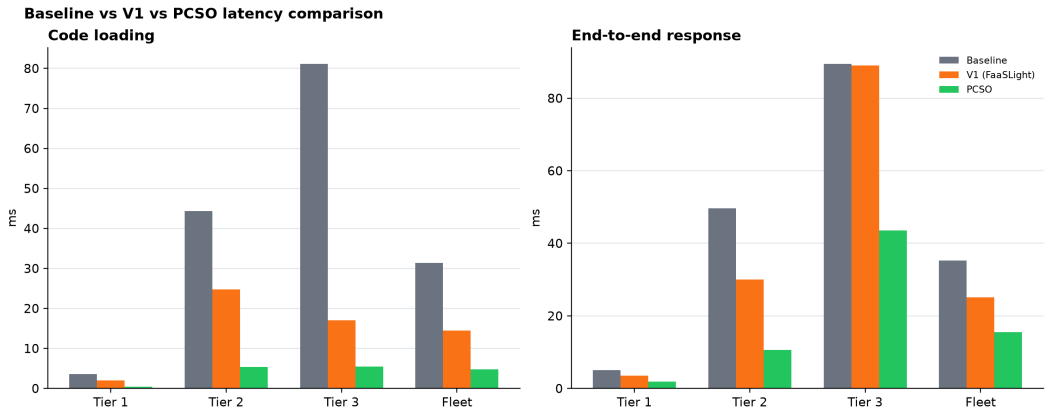


Figure 4.1: Mean code-loading and end-to-end response latency for the eager baseline, V1/FaaSLight, and final PCSO.

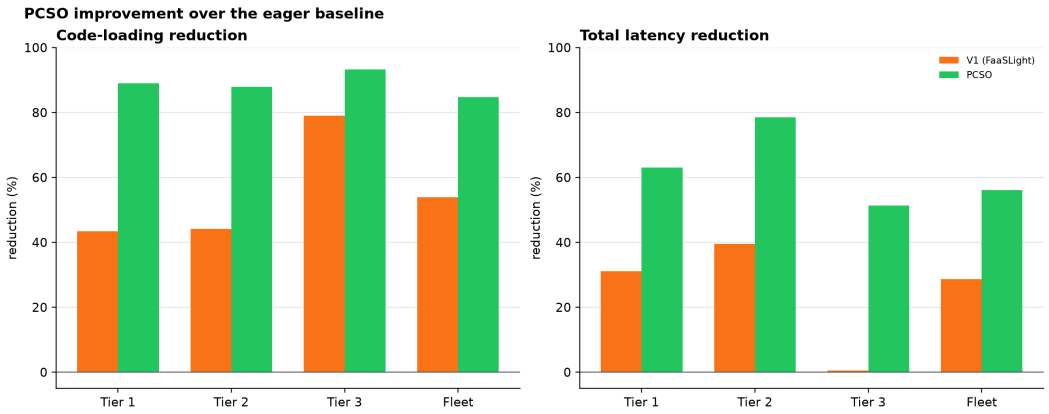


Figure 4.2: Reduction relative to the true eager baseline. PCSO extends the V1 improvement by removing shifted handler-side import cost.

Table 4.3: Mean latency comparison against the true no-optimizer baseline.

	Code loading (ms)			Total response (ms)		
Workload	**Baseline**	**V1**	**PCSO**	**Baseline**	**V1**	**PCSO**
Tier 1	3.62	2.05	0.40	5.08	3.50	1.88
Tier 2	44.32	24.76	5.37	49.61	30.02	10.63
Tier 3	81.15	17.05	5.46	89.47	88.98	43.52
Blended fleet	31.36	14.45	4.77	35.29	25.16	15.48

4.3 Tier 3 analysis

V1 reduces Tier 3 code-loading latency from 81.15 ms to 17.05 ms, but the heavy OCR, machine-learning, and media dependencies are then imported inside the handler. Consequently, total latency changes only from 89.47 ms to 88.98 ms. PCSO uses the observed transition history to warm the predicted heavy dependency closure before the request. Total Tier 3 latency therefore falls to 43.52 ms, a 51.4% reduction relative to the baseline and a 51.1% reduction relative to V1.

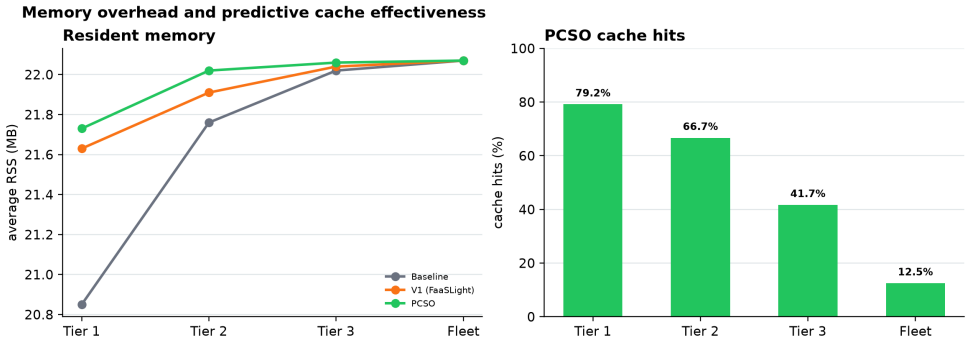


Figure 4.3: Resident memory and PCSO predictive cache-hit rate across the workload profiles.

4.4 Blended fleet analysis

For the blended fleet, V1 reduces code-loading latency by 53.9% and total latency by 28.7%. PCSO improves those reductions to 84.8% and 56.1%. The tiered sequences produce higher prediction hit rates than the deliberately mixed fleet: 79.2% for Tier 1, 66.7% for Tier 2, 41.7% for Tier 3, and 12.5% for the blended profile.

Chapter 5

CONCLUSIONS AND FUTURE WORK

5.1 Conclusion

PCSO combines static dependency analysis, transition prediction, and dependency-closure preloading. Against the true eager baseline, it reduces blended code-loading latency by 84.8% and total response latency by 56.1%. The three-way comparison clarifies the source of the gain: V1 removes optional imports from startup, while PCSO prevents heavy deferred imports from reappearing on the handler's critical path.

5.2 Limitations and future work

The evaluation is a controlled application-level benchmark rather than a measurement of a particular cloud provider's network, billing, or container scheduler. The dependency weights model relative initialization cost and should not be interpreted as provider-specific timings. The predictor is first-order and online; it can be inaccurate after an abrupt workload change. Future work should evaluate production traces, add confidence decay, enforce a memory budget, and move preloading to an asynchronous worker with explicit resource limits.

REFERENCES

[1] Y. Liu *et al.*, "FaaSLight: General Application-Level Cold-Start Latency Optimization for Function-as-a-Service," arXiv:2207.08175, 2023.

[2] Amazon Web Services, "Understanding AWS Lambda scaling and cold starts," AWS documentation.